TOPIC 1

Polynomial Regression: Explained Slides

The original class slides, with a plain explanation under each one

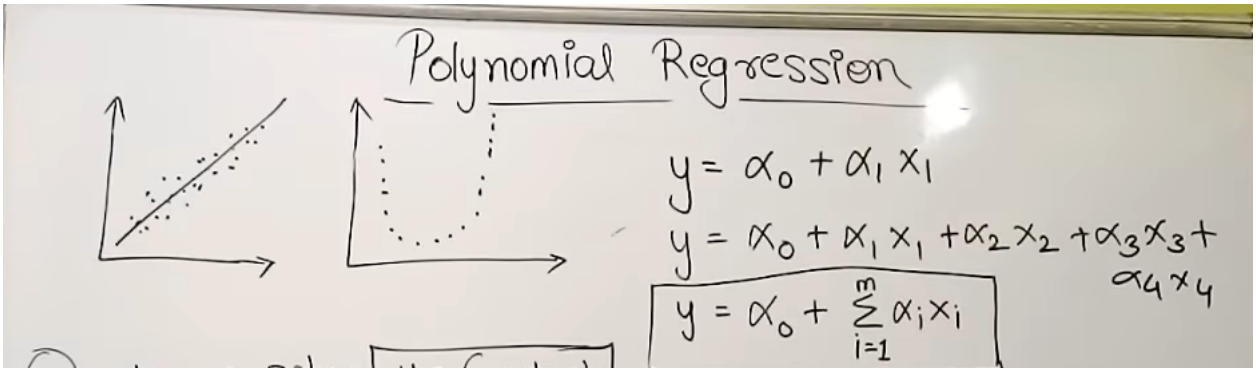
> **NOTE** **How to read this document**
>
> - Every original slide is kept exactly as it was, with its slide number under it
> - Under each slide (or small set of slides) you get a plain explanation: **what you are looking at** and **what it means**
> - The slides come from a recorded class, so a few look like screenshots of a video: that is normal

Straight-line (linear) regression works well when the points roughly follow a line. Many real relationships **bend**: salary grows faster and faster with seniority, for example. **Polynomial regression** keeps the same easy machinery as linear regression but lets the fitted line **curve**. These 12 slides show the idea step by step: the equations, why a straight line fails on curved data, how new columns (squares of the features) fix it, and the salary example used in the class code.

Words you will meet

Word	Plain meaning
Polynomial	An expression with powers of x, such as x, x^2 and x^3
Degree	The highest power of x used. Degree 1 is a straight line, degree 2 a parabola (a U shape), and so on
Feature	An input column (x_1, x_2 ...) the model learns from
Coefficient (α or b)	A number the model learns. Each feature gets one; α_0 / b_0 is the starting value (intercept)
Intercept	The value of y when all features are 0 (α_0 or b_0)
Linear in the coefficients	The model only adds up coefficient × feature terms. This is why a curve can still be fitted with linear regression
Underfitting	The model is too simple for the data (a straight line through a U shape)

Slide 1: From a line to many features

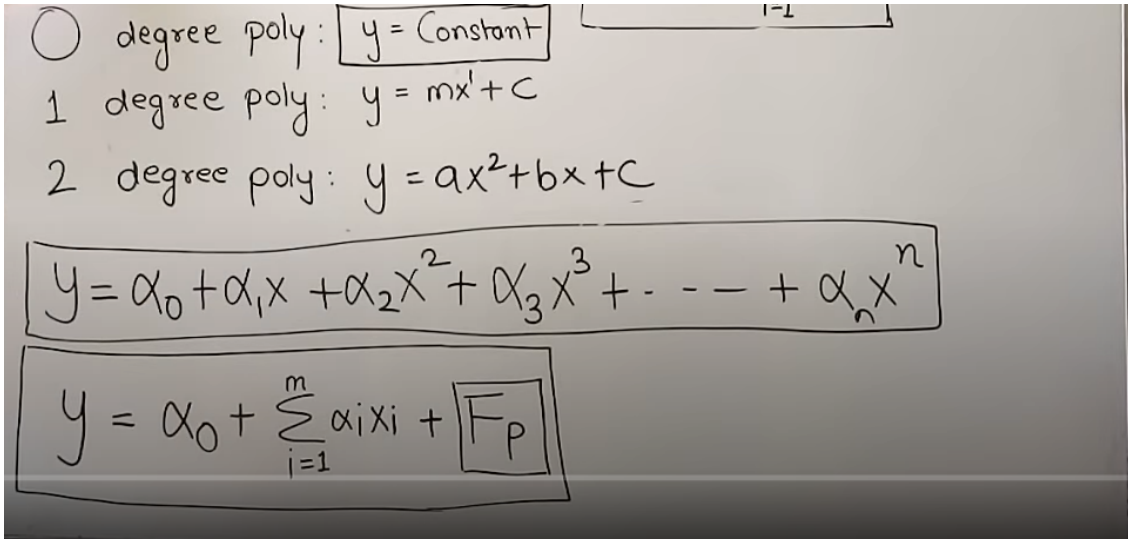


Slide 1

The title slide shows two small sketches and three equations. The left sketch is data that follows a **straight line**; the right sketch is data shaped like a **U**, which a straight line cannot follow. The equations grow step by step: first one input with one coefficient, then four inputs (x_1 to x_4), then the same thing written compactly with the summation sign Σ (add up the terms for i = 1 to m).

- **$y = \alpha_0 + \alpha_1 x_1$** : a straight line. α_0 is where the line starts, α_1 is how steeply it rises
- **$y = \alpha_0 + \alpha_1 x_1 + \alpha_2 x_2 + \alpha_3 x_3 + \alpha_4 x_4$** : several input features, each with its own coefficient (this is multiple linear regression)
- **$y = \alpha_0 + \Sigma\, \alpha_i x_i$** : the same equation in short form

Slide 2: What is a polynomial of degree n?



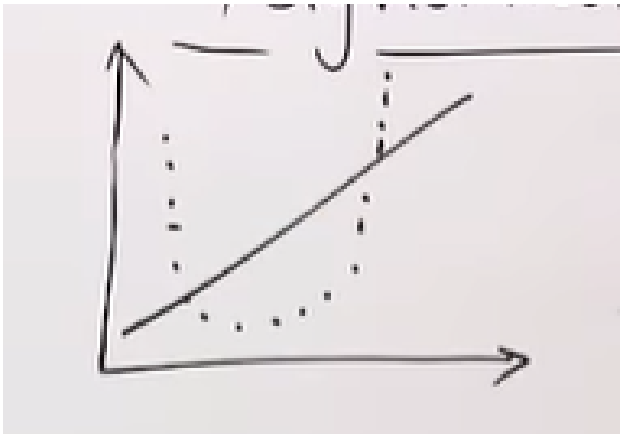
Slide 2

The **degree** is the highest power of x in the equation. The slide lists them from simplest to most flexible:

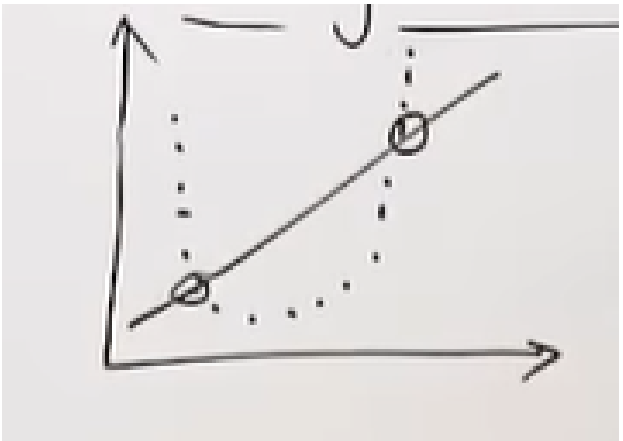
- **Degree 0:** y = constant. A flat line
- **Degree 1:** y = mx + c. A straight line
- **Degree 2:** $y = ax^2 + bx + c$. A single smooth bend (a U or an upside-down U)
- **Degree n:** $y = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \alpha_3 x^3 + \ldots + \alpha_n x^n$. More terms allow more bends

$$y = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \alpha_3 x^3 + \ldots + \alpha_n x^n$$

Slides 3 to 4: Why a straight line fails on curved data



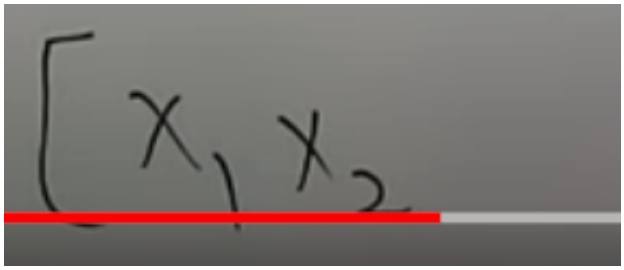
Slide 3



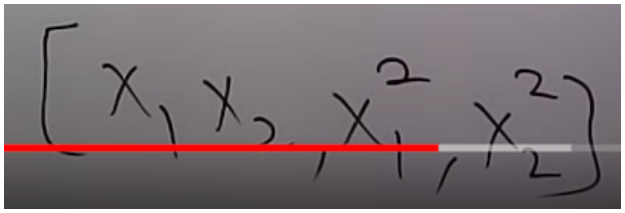
Slide 4

Here the same U-shaped dots are fitted with a **straight line**. In slide 3 the line cuts across the dots; in slide 4 the two circles mark the **only places where the line actually touches the data**. Everywhere else the line is far away from the points, so its predictions are poor. This is **underfitting**: the model is too simple for the shape of the data. Fitting a straight line to curved data gives large errors no matter how well it is trained.

Slides 5 to 6: The trick: add new columns



Slide 5

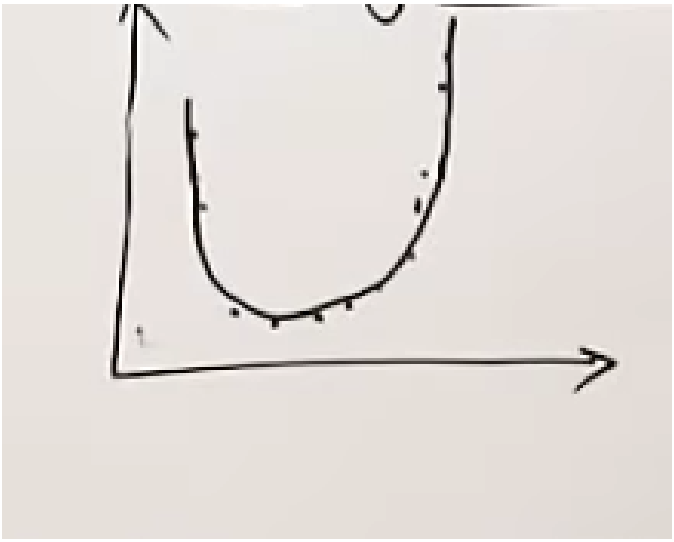


Slide 6

Slide 5 shows an ordinary row of input features, **[x_1, x_2]**. Slide 6 shows the same row after we **add the square of each feature**: **[x_1, x_2, x_1^2, x_2^2]**. Nothing else changes: we simply create extra columns. A straight-line model that now sees x_1^2 and x_2^2 as inputs can draw a bend. This is exactly what scikit-learn's PolynomialFeatures does. (For degree 2 and several features it can also add products such as $x_1 \cdot x_2$; the slide shows the squares only.)

- **Before:** the model learns from [x_1, x_2]
- **After:** the model learns from [x_1, x_2, x_1^2, x_2^2], then ordinary linear regression is used on these new columns

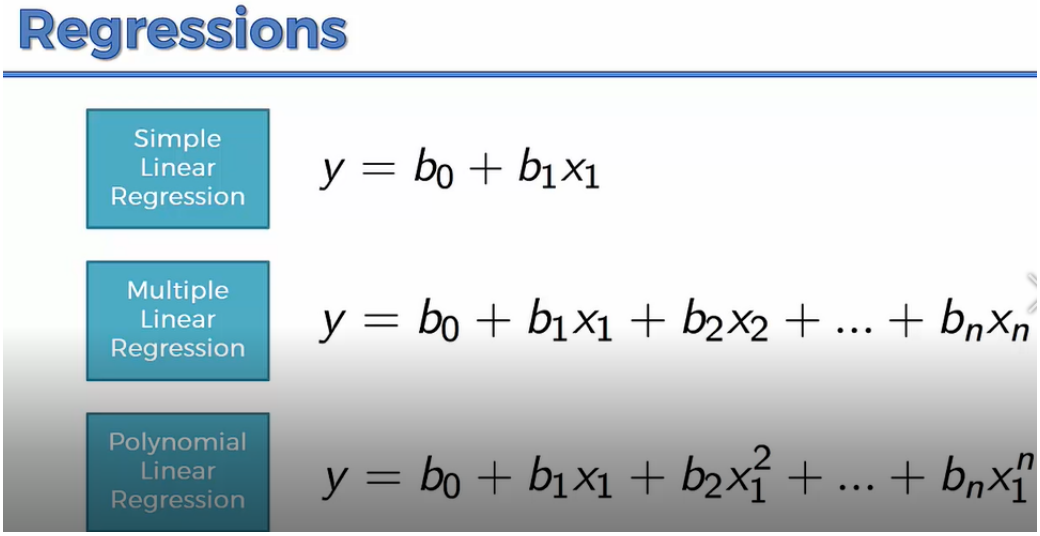
Slide 7: The curve now follows the data



Slide 7

With the extra squared column the fitted line becomes a **curve** that passes close to the U-shaped dots. Compare this with slides 3 and 4, where the straight line missed most of the points.

Slide 8: The three regressions side by side

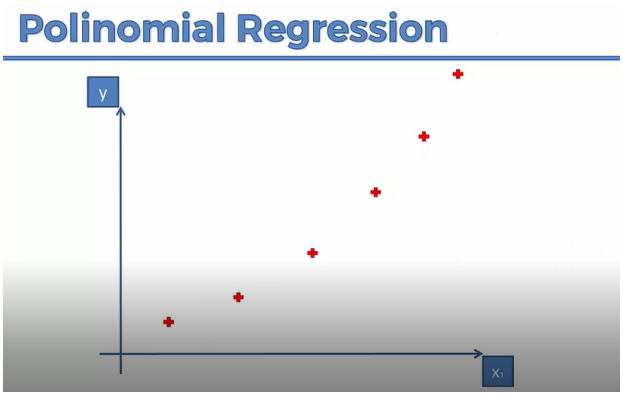


Slide 8

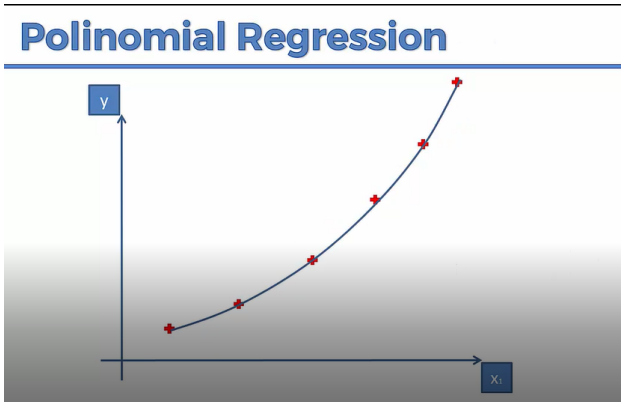
This course slide lines up the three models you have now met. It uses b for the coefficients (the previous slides used α; they mean the same thing). Only the highlighted new idea is different: **powers of one feature** are added.

- **Simple linear:** $y = b_0 + b_1 x_1$. One feature, a straight line
- **Multiple linear:** $y = b_0 + b_1 x_1 + b_2 x_2 + \ldots + b_n x_n$. Several different features, still a straight (flat) relationship
- **Polynomial:** $y = b_0 + b_1 x_1 + b_2 x_1^2 + \ldots + b_n x_1^n$. **One feature raised to higher powers**, which allows a curve

Slides 9 to 10: Data that curves upward, and the curve that fits it



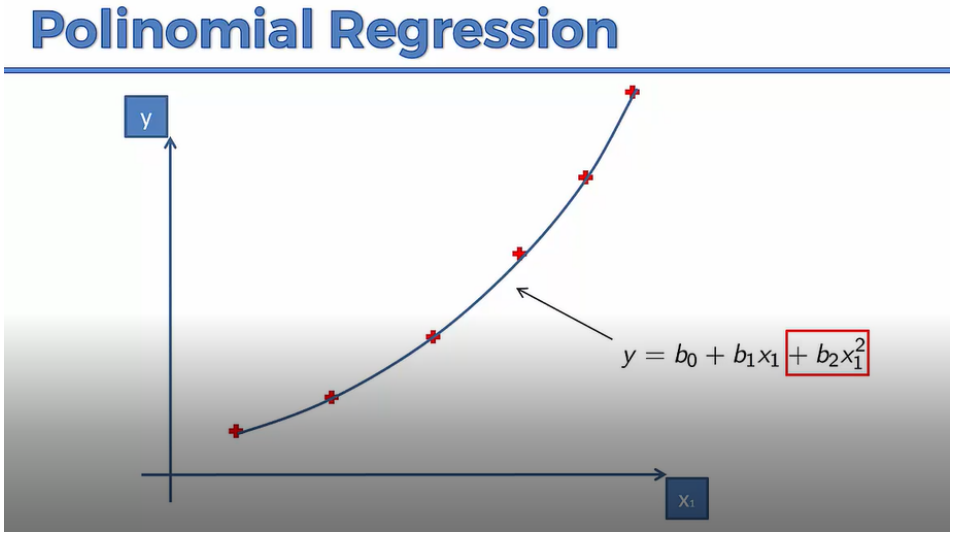
Slide 9



Slide 10

Slide 9 shows red crosses: the data points of y plotted against x_1. They climb slowly at first and then faster and faster, so a straight line would be a poor choice. Slide 10 draws a smooth **curve through the points**. That curve is the polynomial regression model.

Slide 11: Which term makes the bend?

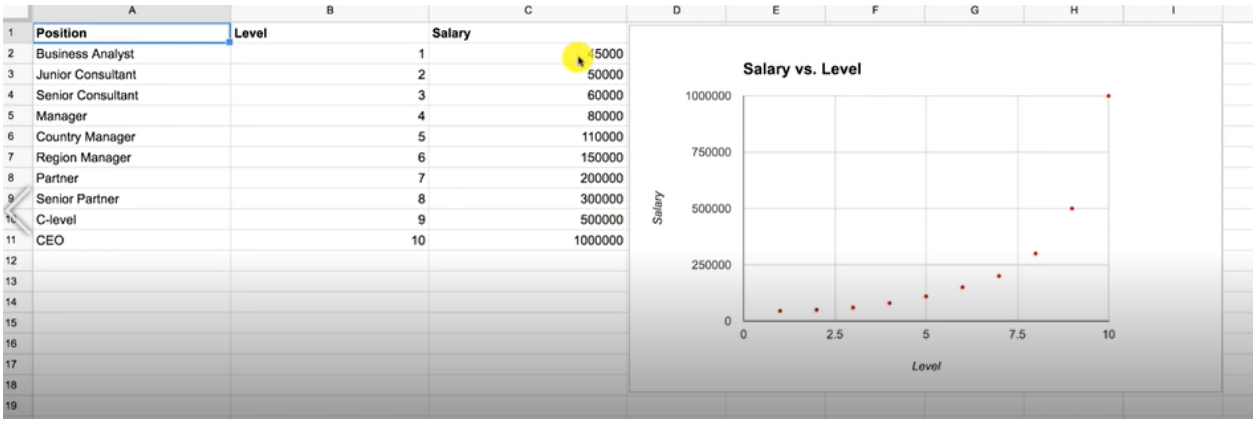


Slide 11

The red box highlights the last term, $b_2x_1^2$. The first two terms, $b_0 + b_1x_1$, alone describe a straight line. Adding the **squared term** is what bends the line into the curve in the picture.

$$y = b_0 + b_1x_1 + b_2x_1^2$$

Slide 12: The salary example used in class



	Position	Level	Salary
2	Business Analyst	1	45000
3	Junior Consultant	2	50000
4	Senior Consultant	3	60000
5	Manager	4	80000
6	Country Manager	5	110000
7	Region Manager	6	150000
8	Partner	7	200000
9	Senior Partner	8	300000
10	C-level	9	500000
11	CEO	10	1000000

Slide 12

This is the data set behind the class code. Each row is a job **position** with its **level** (1 to 10) and **salary**. The scatter plot on the right (Salary vs Level) shows that salary stays low for the first levels and then shoots up for the top levels, a curved relationship, so polynomial regression is a natural choice. The first salary is partly hidden by the mouse pointer; the class file emp_sal.csv holds the exact numbers. The job titles on this screenshot differ slightly from the titles in that file, but levels and salaries match.

- **Input (X):** Level
- **Output (y):** Salary
- **Goal in the class code:** predict the salary for a level between 6 and 7 (level 6.5)